

Taller 3 — Documento de explicación taller 3

Matemáticas Discretas I — Criptografía, grafos, álgebra de Boole, Shannon y computación cuántica

Maria Paula Pérez Meléndez

July 30, 2026

Introducción

Este documento acompaña el repositorio `taller3_MDI` y responde, para cada uno de los 10 ejercicios, las cinco preguntas pedidas en las instrucciones del taller: qué problema resuelve el programa, qué idea matemática usa, cómo se ejecuta, qué pruebas se hicieron y qué limitaciones tiene la solución.

Todo el código fue escrito en **Python**, sin librerías externas obligatorias (solo se usa `numpy` en el punto 10, para el manejo de vectores y números aleatorios). Cada programa se ejecuta descargando los `.py` desde la raíz del repositorio con:

```
python3 src/<carpeta>/punto_<n>.py
```

y pide los datos por consola (`input`).

Bloque A. Criptografía

Punto 1. Cifrado César (`src/cripto/punto_1.py`)

Qué problema resuelve. Cifra y descifra un mensaje desplazando cada letra una cantidad fija k de posiciones en el alfabeto, y además permite romper un mensaje cifrado probando los 26 desplazamientos posibles cuando no se conoce k .

Idea matemática. El cifrado César es una función de desplazamiento módulo 26 sobre el alfabeto $\{A, \dots, Z\}$. Si a cada letra se le asigna un número $0, \dots, 25$ según su posición, cifrar es la operación

$$C = (P + k) \bmod 26,$$

y descifrar es la operación inversa

$$P = (C - k) \bmod 26 = (C + (-k)) \bmod 26.$$

Por eso el código simplemente llama a `cifrar(texto, -desplazamiento)` para descifrar: sumar $-k$ módulo 26 deshace el corrimiento de $+k$. Los caracteres que no pertenecen al alfabeto (espacios, números, signos) se devuelven sin cambios, y se conserva si la letra original era mayúscula o minúscula.

Cómo se ejecuta.

```
python3 src/cripto/punto_1.py
```

El programa muestra un menú con 4 opciones: cifrar, descifrar, fuerza bruta y salir.

Qué pruebas se hicieron.

```
===== CIFRADO CÉSAR =====  
1. Cifrar  
2. Descifrar  
3. Fuerza bruta  
4. Salir  
  
Seleccione una opción: 1  
Ingrese el texto: HOLA UNAL  
Ingrese el desplazamiento: 3  
  
Texto cifrado:  
KROD XQDO
```

Figure 1: cifrar.

```
===== CIFRADO CÉSAR =====  
1. Cifrar  
2. Descifrar  
3. Fuerza bruta  
4. Salir  
  
Seleccione una opción: 2  
Ingrese el texto cifrado: KROD XQDO  
Ingrese el desplazamiento: 3  
  
Texto descifrado:  
HOLA UNAL
```

Figure 2: descifrar.

===== CIFRADO CÉSAR =====

1. Cifrar
2. Descifrar
3. Fuerza bruta
4. Salir

Seleccione una opción: 3

Ingrese el texto cifrado: KROD XQDO

Posibles descifrados:

k = 0: KROD XQDO
k = 1: JQNC WPCN
k = 2: IPMB VOBM
k = 3: HOLA UNAL
k = 4: GNKZ TMZK
k = 5: FMJY SLYJ
k = 6: ELIX RKXI
k = 7: DKHW QJWH
k = 8: CJGV PIVG
k = 9: BIFU OHUF
k = 10: AHET NGTE
k = 11: ZGDS MFSD
k = 12: YFCR LERC
k = 13: XEBQ KDQB
k = 14: WDAP JCPA
k = 15: VCZO IBOZ
k = 16: UBYN HANY
k = 17: TAXM GZMX
k = 18: SZWL FYLW
k = 19: RYVK EXKV
k = 20: QXUJ DWJU
k = 21: PWTI CVIT
k = 22: OVSH BUHS
k = 23: NURG ATGR
k = 24: MTQF ZSFQ
k = 25: LSPE YREP

Figure 3: no se sabe k.

Limitaciones. El alfabeto solo cubre A–Z (sin ñ ni tildes); un carácter con tilde se trata como símbolo no reconocido y se deja igual. Al ser un cifrado monoalfabético, es vulnerable a fuerza bruta (solo 26 llaves posibles) y a análisis de frecuencias, por lo que no debe usarse como mecanismo de seguridad real, solo como ejemplo didáctico.

Punto 2. RSA de juguete (`src/cripto/punto_2.py`)

Qué problema resuelve. Genera una llave RSA pequeña a partir de dos primos p, q y un exponente público e , y usa esa llave para cifrar y descifrar un mensaje numérico M .

Idea matemática. RSA se apoya en aritmética modular y en el teorema de Euler. A partir de $n = pq$ y $\varphi(n) = (p-1)(q-1)$ se busca d tal que

$$e \cdot d \equiv 1 \pmod{\varphi(n)},$$

es decir, d es el inverso modular de e módulo $\varphi(n)$. Ese inverso se calcula con el **algoritmo extendido de Euclides** (función `inverso_modular`), que además de calcular $\gcd(e, \varphi(n))$ encuentra los coeficientes de Bézout. Si $\gcd(e, \varphi(n)) \neq 1$, el inverso no existe y el programa lo reporta como error. El cifrado y el descifrado son exponenciación modular:

$$C \equiv M^e \pmod{n}, \quad M \equiv C^d \pmod{n},$$

calculada de forma eficiente con `pow(base, exp, mod)` (exponenciación rápida por cuadrados), en vez de elevar el número completo y luego tomar el módulo.

Cómo se ejecuta.

```
python3 src/cripto/punto_2.py
```

Pide p, q, e y el mensaje M por consola.

Qué pruebas se hicieron. Caso obligatorio del enunciado, $p = 61, q = 53, e = 17, M = 65$:

```
RSA DE JUGUETE  
Ingrese p: 61  
Ingrese q: 53  
Ingrese e: 17  
Ingrese el mensaje: 65
```

```
RESULTADOS  
n = 3233  
phi(n) = 3120  
d = 2753  
Cifrado = 2790  
Descifrado = 65
```

Figure 4: prueba 1

RSA DE JUGUETE

Ingrese p: 19

Ingrese q: 29

Ingrese e: 5

Ingrese el mensaje: 57

RESULTADOS

$n = 551$

$\phi(n) = 504$

$d = 101$

Cifrado = 57

Descifrado = 57

Figure 5: prueba 2

```
RSA DE JUGUETE
Ingrese p: 11
Ingrese q: 13
Ingrese e: 7
Ingrese el mensaje: 9

RESULTADOS
n = 143
phi(n) = 120
d = 103
Cifrado = 48
Descifrado = 9
```

Figure 6: prueba 3

También se probó con un e no coprimo con $\varphi(n)$ (por ejemplo $e = 6$ con el mismo p, q), y el programa responde correctamente: *“Error: e no es válido porque no es coprimo con $\phi(n)$.”*

Limitaciones. Es una versión “de juguete”: usa primos pequeños que se pueden factorizar con facilidad, por lo que no ofrece seguridad real (RSA real usa primos de cientos de dígitos). Tampoco valida que p y q sean primos ni que $M < n$; si $M \geq n$ el resultado del descifrado no coincidiría con el mensaje original por las propiedades del módulo.

Punto 3. MPC básico: suma secreta con tres servidores (src/cripto/punto_3.py)

Qué problema resuelve. Simula un protocolo de computación multipartita segura (MPC) en el que se calcula la suma y el promedio de una lista de notas sin que ningún servidor individual conozca las notas originales.

Idea matemática. Cada nota x se reparte en tres partes aleatorias módulo M tales que

$$x \equiv s_1 + s_2 + s_3 \pmod{M}.$$

El programa genera s_1 y s_2 aleatoriamente en $[0, M - 1]$ y calcula la tercera parte como

$$s_3 = (x - s_1 - s_2) \bmod M,$$

de modo que la suma de las tres partes, módulo M , siempre reconstruye x . Como s_1 y s_2 son uniformemente aleatorios, cada parte por separado no da ninguna información sobre x (es un esquema de *secret sharing* aditivo). Cada “servidor” solo recibe una de las tres partes de cada nota; al sumar las partes correspondientes de cada servidor y tomar módulo M se reconstruye la suma total de las notas, sin que ningún servidor haya visto una nota completa.

Cómo se ejecuta.

```
python3 src/cripto/punto_3.py
```

Pide las notas separadas por espacios.

Qué pruebas se hicieron. Ejemplo mínimo del enunciado, notas [40, 35, 50, 25]:

```
Ingrese las notas separadas por espacios: 40 35 50 25
Servidor 1: [794231, 280742, 510682, 75427]
Servidor 2: [338677, 431795, 612879, 874299]
Servidor 3: [867138, 287501, 876495, 50302]

Resultados
Suma: 150
Promedio: 37.5
```

Figure 7: prueba 1

```
Ingrese las notas separadas por espacios: 10 20 30
Servidor 1: [218068, 445293, 353647]
Servidor 2: [11013, 448119, 57527]
Servidor 3: [770932, 106611, 588859]

Resultados
Suma: 60
Promedio: 20.0
```

Figure 8: prueba 2

```

Ingrese las notas separadas por espacios: 45 18 32 40 27 15 50 8 10 35
Servidor 1: [486456, 513345, 984514, 891956, 558873, 854765, 854616, 479112, 611273, 278880]
Servidor 2: [779052, 119333, 123837, 848330, 98797, 37975, 658011, 780332, 439101, 508553]
Servidor 3: [734543, 367343, 891687, 259760, 342360, 107278, 487429, 740570, 949642, 212605]

Resultados
Suma: 280
Promedio: 28.0

```

Figure 9: prueba 3

Se verificó que la suma reconstruida es 150 y el promedio 37.5, tal como pide el enunciado, aunque ninguno de los tres servidores contiene individualmente los valores 40, 35, 50 o 25.

Limitaciones. El programa reconstruye la suma dentro del mismo proceso (no hay comunicación real entre servidores separados), por lo que es una simulación pedagógica de la idea, no una implementación distribuida real. Además, no se protege contra colusión: si dos de los tres servidores comparten sus partes, pueden despejar la tercera y recuperar la nota original.

Bloque B. Grafos

Punto 4. Ruta más corta con Dijkstra (src/grafos/punto_4.py)

Qué problema resuelve. Dado un grafo ponderado que representa una red de transporte, encuentra la distancia mínima y la ruta más corta desde un nodo de origen hacia todos los demás nodos.

Idea matemática. Se implementa el **algoritmo de Dijkstra** con una cola de prioridad (**heapq**). La idea central es la propiedad de optimalidad de subcaminos: si el camino más corto de A a C pasa por B, entonces el tramo de A a B también debe ser el más corto posible. El algoritmo mantiene un vector de distancias tentativas (inicializadas en infinito, salvo el origen en 0) y, en cada paso, extrae de la cola el nodo no procesado con menor distancia tentativa, y *relaja* sus aristas: si llegar a un vecino pasando por el nodo actual mejora la distancia conocida, se actualiza. Esto solo es válido si **todos los pesos son no negativos**, porque el algoritmo asume que una vez que un nodo es extraído con distancia mínima, ya no puede aparecer un camino más corto hacia él (con pesos negativos esa garantía se rompe).

Cómo se ejecuta.

```
python3 src/grafos/punto_4.py
```

Pide el número de nodos, sus nombres, el número de aristas, cada arista como **origen destino peso**, y el nodo de origen.

Qué pruebas se hicieron. Grafo de prueba con 8 nodos y 12 aristas

Arista 11:

a h 10

Arista 12:

h f 9

Nodo origen: a

DISTANCIAS MÍNIMAS

A -> A: 0

A -> B: 3.0

A -> C: 5.0

A -> D: 11.0

A -> E: 6.0

A -> F: 12.0

A -> G: 14.0

A -> H: 10.0

CAMINOS

A -> A: A

A -> B: A -> B

A -> C: A -> C

A -> D: A -> B -> D

A -> E: A -> C -> E

A -> F: A -> C -> F

A -> G: A -> C -> G

A -> H: A -> H

Figure 10: prueba 1

Ingrese el número de nodos8

Ingrese el nombre de cada nodo:

Nodo: a

Nodo: b

Nodo: c

Nodo: d

Nodo: e

Nodo: f

Nodo: g

Nodo: h

Ingrese el número de aristas: 12

Figure 11: prueba 1

```
Ingrese las aristas en el formato:  
origen destino peso
```

```
Arista 1:
```

```
a c 5
```

```
Arista 2:
```

```
c f 7
```

```
Arista 3:
```

```
c e 1
```

```
Arista 4:
```

```
b e 4
```

```
Arista 5:
```

```
e g 15
```

```
Arista 6:
```

```
c h 9
```

```
Arista 7:
```

```
c g 9
```

```
Arista 8:
```

```
b d 8
```

```
Arista 9:
```

```
b c 5
```

```
Arista 10:
```

```
a b 3
```

Figure 12: prueba 1

Limitaciones. Solo admite pesos no negativos (si se ingresa un peso negativo el resultado ya no está garantizado). El grafo se construye como no dirigido (cada arista se agrega en ambos sentidos); para un grafo dirigido habría que modificar `ingreso_grafo`. La entrada es completamente manual

por consola, no se lee desde un archivo externo.

Punto 5. Cierre de una estación (src/grafos/punto_5.py)

Qué problema resuelve. Mide el impacto de cerrar un nodo (estación) o una arista (conexión) de la red: compara las distancias más cortas entre varios pares origen-destino antes y después del cierre.

Idea matemática. Reutiliza Dijkstra (mismo algoritmo del punto 4) ejecutado dos veces: una sobre una copia del grafo original (`deepcopy`) y otra sobre el grafo modificado tras eliminar el nodo o la arista indicada. Para cada par origen-destino se comparan las dos distancias y se clasifica el resultado en una de cuatro categorías: sin cambio, más larga, desconectado (existía camino y ahora no) o ya estaba desconectado. Eliminar un nodo implica quitarlo del diccionario del grafo y borrar todas las referencias a él en las listas de adyacencia de sus vecinos, para que el grafo quede consistente.

Cómo se ejecuta.

```
python3 src/grafos/punto_5.py
```

Pide el grafo igual que en el punto 4, luego mínimo 5 pares origen-destino a evaluar, y finalmente qué se desea cerrar (un nodo, una arista, o nada).

Qué pruebas se hicieron. Usando el mismo grafo de 8 nodos del punto 4, se probaron 5 pares

```
¿Qué desea cerrar?
1. Una estación (nodo)
2. Una conexión (arista)
3. No cerrar nada

Seleccione una opción: 1
Ingrese la estación a cerrar: c

===== TABLA COMPARATIVA (ANTES vs. DESPUÉS DEL CIERRE) =====
```

Origen	Destino	Dist. antes	Dist. despues	Diferencia	Estado
A	F	12.0	19.0	7.0	MAS LARGA
A	G	14.0	22.0	8.0	MAS LARGA
B	H	13.0	13.0	0	SIN CAMBIO
C	D	13.0	INF	-	DESCONECTADO
E	H	10.0	17.0	7.0	MAS LARGA

Figure 13: prueba 1

```
===== TABLA COMPARATIVA (ANTES vs. DESPUÉS DEL CIERRE) =====
```

Origen	Destino	Dist. antes	Dist. despues	Diferencia	Estado
A	C	2.0	2.0	0	SIN CAMBIO
A	D	3.0	3.0	0	SIN CAMBIO
B	D	2.0	2.0	0	SIN CAMBIO
A	B	1.0	1.0	0	SIN CAMBIO
C	D	1.0	1.0	0	SIN CAMBIO

Figure 14: prueba 2

```
===== TABLA COMPARATIVA (ANTES vs. DESPUÉS DEL CIERRE) =====
```

Origen	Destino	Dist. antes	Dist. despues	Diferencia	Estado
A	C	2.0	6.0	4.0	MAS LARGA
A	D	3.0	5.0	2.0	MAS LARGA
B	D	2.0	6.0	4.0	MAS LARGA
A	B	1.0	1.0	0	SIN CAMBIO
C	D	1.0	1.0	0	SIN CAMBIO

Figure 15: prueba 3

Limitaciones. Solo se puede cerrar un nodo o una arista a la vez por ejecución (no cierres simultáneos). Al igual que en el punto 4, el grafo es no dirigido y los pesos deben ser no negativos. Si un par origen-destino queda desconectado, se reporta como INF, pero el programa no sugiere rutas alternas fuera del grafo.

Punto 6. Coloreo de grafos (src/grafos/punto_6.py)

Qué problema resuelve. Asigna franjas horarias (colores) a un conjunto de materias de modo que dos materias con estudiantes en común (es decir, conectadas por una arista de conflicto) nunca queden en la misma franja.

Idea matemática. Es un problema de **coloreo de grafos**: cada vértice (materia) debe recibir un color (horario) distinto al de todos sus vecinos. Se implementa un **algoritmo voraz (greedy)**: se recorren los vértices en el orden en que fueron ingresados y, para cada uno, se asigna el menor color disponible que no esté siendo usado por ninguno de sus vecinos ya coloreados. El algoritmo garantiza que la coloración es *válida* (nunca colorea igual a dos vértices adyacentes), pero no garantiza el número mínimo de colores posible (el número cromático), porque ese orden greedy puede “desperdiciar” colores si el orden de los vértices no es el óptimo; encontrar el número cromático exacto es, en general, un problema NP-difícil.

Cómo se ejecuta.

```
python3 src/grafos/punto_6.py
```

Pide el número de materias, sus nombres, el número de conflictos y cada conflicto como `Materia1` `Materia2`.

Qué pruebas se hicieron. Grafo de prueba con 10 materias
y conflictos:

```
Ingrese el número de materias: 10

Ingrese el nombre de cada materia:
Materia: CALCULO
Materia: DISCRETAS
Materia: FISICA
Materia: algebra
Materia: programacion
Materia: bases
Materia: ingles
Materia: estadistica
Materia: probabilidad
Materia: quimica
```

Figure 16: prueba 1


```
Ingrese el número de conflictos: 12

Ingrese cada conflicto (Materia1 Materia2):
Conflicto 1:
calculo discretas
Conflicto 2:
fisica calculo
Conflicto 3:
programacion bases
Conflicto 4:
bases probabilidad
Conflicto 5:
quimica estadistica
Conflicto 6:
ingles quimica
Conflicto 7:
fisica quimica
Conflicto 8:
algebra discretas
Conflicto 9:
ingles bases
Conflicto 10:
calculo quimica
Conflicto 11:
ingles fisica
Conflicto 12:
DISCRETAS PROGRAMACION

Coloreo válido.
```

Figure 17: prueba 1

```
===== HORARIOS =====
```

```
Horario 1: CALCULO, ALGEBRA, PROGRAMACION, INGLES, ESTADISTICA, PROBABILIDAD
```

```
Horario 2: DISCRETAS, FISICA, BASES
```

```
Horario 3: QUIMICA
```

Figure 18: prueba 1

ASIGNACIÓN DE HORARIOS

Ingrese el número de materias: 10

Ingrese el nombre de cada materia:

Materia: multivariado

Materia: fisicaII

Materia: discretasI

Materia: discretasII

Materia: fisicaI

Materia: diferencial

Materia: integral

Materia: lineal

Materia: inglesI

Materia: inglesII

Figure 19: prueba 2

```
Ingrese el número de conflictos: 9

Ingrese cada conflicto (Materia1 Materia2):
Conflicto 1:
multivariado fisicaII
Conflicto 2:
fisicaII discretasI
Conflicto 3:
discretasI discretasII
Conflicto 4:
discretasII fisicaI
Conflicto 5:
fisicaI diferencial
Conflicto 6:
diferencial integral
Conflicto 7:
integral lineal
Conflicto 8:
lineal inglesI
Conflicto 9:
inglesI inglesII
```

Figure 20: prueba 2

```
Coloreo válido.

===== HORARIOS =====

Horario 1: MULTIVARIADO, DISCRETASI, FISICAI, INTEGRAL, INGLESII
Horario 2: FISICAII, DISCRETASII, DIFERENCIAL, LINEAL, INGLESII

Total de horarios utilizados: 2
```

Figure 21: prueba 2

ASIGNACIÓN DE HORARIOS

Ingrese el número de materias: 10

Ingrese el nombre de cada materia:

Materia: elementos

Materia: fisicaI

Materia: integral

Materia: bases

Materia: estructuras

Materia: inglesI

Materia: discretasII

Materia: discretasI

Materia: lineal

Materia: multivariado

Figure 22: prueba 3

```

Ingrese el número de conflictos: 9

Ingrese cada conflicto (Materia1 Materia2):
Conflicto 1:
elementos fisicaI
Conflicto 2:
elementos integral
Conflicto 3:
elementos bases
Conflicto 4:
elementos estructuras
Conflicto 5:
elementos inglesI
Conflicto 6:
elementos discretasI
Conflicto 7:
elementos discretasII
Conflicto 8:
elementos lineal
Conflicto 9:
elementos multivariado

```

Figure 23: prueba 3

```

Coloreo válido.

===== HORARIOS =====

Horario 1: ELEMENTOS
Horario 2: FISICAI, INTEGRAL, BASES, ESTRUCTURAS, INGLESII, DISCRETASII, DISCRETASI, LINEAL, MULTIVARIADO

```

Figure 24: prueba 3

La función `verificar` confirmó que ningún par de materias adyacentes quedó en el mismo horario.

Limitaciones. Al ser voraz y depender del orden de ingreso de los vértices, el número de colores usados puede no ser el mínimo teórico; con otro orden de ingreso del mismo grafo se podría obtener una cantidad distinta de horarios. El programa no intenta minimizar colores probando distintos órdenes.

Bloque C. Álgebra de Boole, Shannon y computación cuántica

Punto 7. Tablas de verdad y circuitos lógicos (src/boole/punto_7.py)

Qué problema resuelve. Genera la tabla de verdad completa de tres expresiones booleanas de 4 variables (A, B, C, D) y permite evaluar una de ellas en una entrada específica escogida por el usuario.

Idea matemática. Una expresión booleana de n variables define una función $\{0, 1\}^n \rightarrow \{0, 1\}$. Como el dominio es finito, su tabla de verdad se puede construir enumerando **todas** las 2^n combinaciones posibles de entrada (con `itertools.product([0,1], repeat=4)`) y evaluando la expresión en cada una. Esa tabla es exactamente la especificación de un circuito lógico combinacional: cada fila corresponde a una combinación de señales de entrada y la columna de resultado a la señal de salida que produciría el circuito construido con compuertas AND, OR, NOT y XOR según la expresión.

Cómo se ejecuta.

```
python3 src/boole/punto_7.py
```

Imprime automáticamente las tres tablas de verdad y luego pide escoger una expresión y los valores de A, B, C, D para evaluarla puntualmente.

Qué pruebas se hicieron.

```
=====
(A AND B) OR (NOT C)
=====
```

A	B	C	D	Resultado
0	0	0	0	1
0	0	0	1	1
0	0	1	0	0
0	0	1	1	0
0	1	0	0	1
0	1	0	1	1
0	1	1	0	0
0	1	1	1	0
1	0	0	0	1
1	0	0	1	1
1	0	1	0	0
1	0	1	1	0
1	1	0	0	1
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1

Figure 25: prueba 1

```
Evaluación de una expresión

Seleccione la expresión (1, 2 o 3): 1
A (0 o 1): 1
B (0 o 1): 1
C (0 o 1): 0
D (0 o 1): 0

Resultado: 1
```

Figure 26: prueba 1

=====				
(A OR B) AND (NOT A OR C)				
=====				
A	B	C	D	Resultado
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	1
0	1	0	1	1
0	1	1	0	1
0	1	1	1	1
1	0	0	0	0
1	0	0	1	0
1	0	1	0	1
1	0	1	1	1
1	1	0	0	0
1	1	0	1	0
1	1	1	0	1
1	1	1	1	1

Figure 27: prueba 2

```

Evaluación de una expresión

Seleccione la expresión (1, 2 o 3): 2
A (0 o 1): 1
B (0 o 1): 0
C (0 o 1): 1
D (0 o 1): 0

Resultado: 1

```

Figure 28: prueba 2

=====				
(A XOR B) AND C				
=====				
A	B	C	D	Resultado
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	1
0	1	1	1	1
1	0	0	0	0
1	0	0	1	0
1	0	1	0	1
1	0	1	1	1
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	0

Figure 29: prueba 3

```

Evaluación de una expresión

Seleccione la expresión (1, 2 o 3): 3
A (0 o 1): 1
B (0 o 1): 0
C (0 o 1): 1
D (0 o 1): 0

Resultado: 1

```

Figure 30: prueba 3

Limitaciones. Las expresiones están fijadas en el código (`expresion1`, `expresion2`, `expresion3`); el programa no permite ingresar una expresión booleana arbitraria como texto para evaluarla dinámicamente. La variable D no se usa en ninguna de las tres expresiones, pero de todas formas aparece en la tabla porque el enunciado pide trabajar con 4 variables.

Punto 8. Simplificación de funciones booleanas y Quine-McCluskey (`src/boole/punto_8.py`)

Qué problema resuelve. Implementa el algoritmo de Quine-McCluskey para simplificar una función booleana de 3 o 4 variables expresada como suma de minterminos. Genera la expresión simplificada en suma de productos (SOP) y verifica su equivalencia comparando la tabla de verdad original contra la simplificada.

Idea matemática. El método se basa en el principio de adyacencia lógica $x \cdot y + x \cdot \bar{y} = x$. A partir de los minterminos en formato binario, el algoritmo combina iterativamente pares de términos que difieren en un solo bit (reemplazando esa posición por un guion “-”) hasta obtener los *primos implicantes*. Posteriormente, identifica los primos implicantes esenciales (aquellos que cubren de forma única a algún mintermino) y utiliza un enfoque voraz para cubrir los minterminos restantes. La validez de la simplificación se demuestra mediante la equivalencia lógica de sus tablas de verdad: $F_{original}(A, B, C, D) \equiv F_{simplificada}(A, B, C, D)$.

Cómo se ejecuta.

```
python3 src/boole/punto_8.py
```

El programa solicita el número de variables (3 o 4) y la lista de minterminos separados por espacios. Luego imprime la tabla original, la selección de primos implicantes, la expresión simplificada, la tabla simplificada y la confirmación de equivalencia.

Qué pruebas se hicieron.

```

¿La función tiene 3 o 4 variables?: 4
Ingrese los minterminos separados por espacios: 0 1 2 3

Tabla de verdad original
A B C D | F
0 0 0 0 | 1
0 0 0 1 | 1
0 0 1 0 | 1
0 0 1 1 | 1
0 1 0 0 | 0
0 1 0 1 | 0
0 1 1 0 | 0
0 1 1 1 | 0
1 0 0 0 | 0
1 0 0 1 | 0
1 0 1 0 | 0
1 0 1 1 | 0
1 1 0 0 | 0
1 1 0 1 | 0
1 1 1 0 | 0
1 1 1 1 | 0

Primos implicantes seleccionados:
00-- -> cubre minterminos [0, 1, 2, 3]

```

Figure 31: prueba 1

Expresión simplificada (suma de productos):

$\neg A \wedge \neg B$

Tabla de verdad simplificada

A	B	C	D	F
0	0	0	0	1
0	0	0	1	1
0	0	1	0	1
0	0	1	1	1
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	0
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	0

Verificación:

✓ Las dos expresiones son equivalentes.

Figure 32: prueba 1

```
¿La función tiene 3 o 4 variables?: 3
Ingrese los minterminos separados por espacios: 1 2 3

Tabla de verdad original
A B C | F
0 0 0 | 0
0 0 1 | 1
0 1 0 | 1
0 1 1 | 1
1 0 0 | 0
1 0 1 | 0
1 1 0 | 0
1 1 1 | 0

Primos implicants seleccionados:
  0-1 -> cubre minterminos [1, 3]
  01- -> cubre minterminos [2, 3]
```

Figure 33: prueba 2

Expresión simplificada (suma de productos):

$\neg A \wedge C \vee \neg A \wedge B$

Tabla de verdad simplificada

A	B	C	F
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	0

Verificación:

✓ Las dos expresiones son equivalentes.

Figure 34: prueba 2

Primos implicantes seleccionados:
0--1 -> cubre minterminos [1, 3, 5, 7]

Expresión simplificada (suma de productos):
 $\neg A \wedge D$

Tabla de verdad simplificada

A	B	C	D	F
0	0	0	0	0
0	0	0	1	1
0	0	1	0	0
0	0	1	1	1
0	1	0	0	0
0	1	0	1	1
0	1	1	0	0
0	1	1	1	1
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	0
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	0

Verificación:

✓ Las dos expresiones son equivalentes.

Figure 35: prueba 3

```

Expresión simplificada (suma de productos):
¬A ∧ ¬D

Tabla de verdad simplificada
A B C D | F
0 0 0 0 | 1
0 0 0 1 | 0
0 0 1 0 | 1
0 0 1 1 | 0
0 1 0 0 | 1
0 1 0 1 | 0
0 1 1 0 | 1
0 1 1 1 | 0
1 0 0 0 | 0
1 0 0 1 | 0
1 0 1 0 | 0
1 0 1 1 | 0
1 1 0 0 | 0
1 1 0 1 | 0
1 1 1 0 | 0
1 1 1 1 | 0

Verificación:
✓ Las dos expresiones son equivalentes.

```

Figure 36: prueba 3

Limitaciones. La fase de cobertura utiliza un algoritmo voraz (*greedy*) en lugar del método exacto de Petrick, lo que en ciertos casos particulares con múltiples coberturas mínimas podría seleccionar una solución subóptima (aunque siempre válida). Además, el programa está restringido estrictamente a funciones de 3 o 4 variables y no contempla condiciones de "no importa" (*don't cares*).

Punto 9. Entropía de Shannon (src/shannon/punto_9.py)

Qué problema resuelve. Calcula la frecuencia, la probabilidad y la entropía de Shannon de dos textos, y compara cuál de los dos tiene mayor incertidumbre en la distribución de sus símbolos.

Idea matemática. La entropía de Shannon mide la incertidumbre promedio (en bits) de una fuente de símbolos:

$$H = - \sum_i p_i \log_2(p_i),$$

donde p_i es la probabilidad empírica de cada símbolo (frecuencia dividida entre el total de caracteres). Un mensaje muy repetitivo tiene una distribución muy concentrada (una probabilidad cercana a 1 y el resto casi 0), lo que hace que H sea baja o incluso 0; un mensaje con símbolos muy variados y equiprobables tiene una H más alta, porque hay más incertidumbre sobre cuál será el siguiente símbolo. Es importante notar que H no depende de la longitud del texto, sino de qué tan pareja o concentrada es la distribución de probabilidades.

Cómo se ejecuta.

```
python3 src/shannon/punto_9.py
```

Pide dos textos por consola.

Qué pruebas se hicieron.

```
Ingrese el primer texto: hola
Ingrese el segundo texto: aaaaaaaaaaaaaa

===== TEXTO 1 =====

Frecuencias y probabilidades
-----
h      Frecuencia:  1 Probabilidad: 0.2500
o      Frecuencia:  1 Probabilidad: 0.2500
l      Frecuencia:  1 Probabilidad: 0.2500
a      Frecuencia:  1 Probabilidad: 0.2500

Entropía = 2.0000 bits

===== TEXTO 2 =====

Frecuencias y probabilidades
-----
a      Frecuencia: 15 Probabilidad: 1.0000

Entropía = 0.0000 bits
```

Figure 37: prueba 1

```

Ingrese el primer texto: qqqqqq
Ingrese el segundo texto: hola mundo

===== TEXTO 1 =====

Frecuencias y probabilidades
-----
q          Frecuencia:  6 Probabilidad: 1.0000

Entropía = 0.0000 bits

===== TEXTO 2 =====

Frecuencias y probabilidades
-----
h          Frecuencia:  1 Probabilidad: 0.1000
o          Frecuencia:  2 Probabilidad: 0.2000
l          Frecuencia:  1 Probabilidad: 0.1000
a          Frecuencia:  1 Probabilidad: 0.1000
' ' (espacio) Frecuencia:  1 Probabilidad: 0.1000
m          Frecuencia:  1 Probabilidad: 0.1000
u          Frecuencia:  1 Probabilidad: 0.1000
n          Frecuencia:  1 Probabilidad: 0.1000
d          Frecuencia:  1 Probabilidad: 0.1000

Entropía = 3.1219 bits

===== COMPARACIÓN =====
El segundo texto tiene mayor entropía.
Contiene una distribución de símbolos más variada.

```

Figure 38: prueba 2

```

Ingrese el primer texto: ARAMSAMSAM
Ingrese el segundo texto: HOLAMUNDO

===== TEXTO 1 =====

Frecuencias y probabilidades
-----
A          Frecuencia:   4 Probabilidad: 0.4000
R          Frecuencia:   1 Probabilidad: 0.1000
M          Frecuencia:   3 Probabilidad: 0.3000
S          Frecuencia:   2 Probabilidad: 0.2000

Entropía = 1.8464 bits

===== TEXTO 2 =====

Frecuencias y probabilidades
-----
H          Frecuencia:   1 Probabilidad: 0.1111
O          Frecuencia:   2 Probabilidad: 0.2222
L          Frecuencia:   1 Probabilidad: 0.1111
A          Frecuencia:   1 Probabilidad: 0.1111
M          Frecuencia:   1 Probabilidad: 0.1111
U          Frecuencia:   1 Probabilidad: 0.1111
N          Frecuencia:   1 Probabilidad: 0.1111
D          Frecuencia:   1 Probabilidad: 0.1111

Entropía = 2.9477 bits

===== COMPARACIÓN =====
El segundo texto tiene mayor entropía.
Contiene una distribución de símbolos más variada.

```

Figure 39: prueba 3

Limitaciones. El cálculo trata cada carácter (incluyendo espacios) como un símbolo independiente, sin considerar la estructura del lenguaje natural (por ejemplo, no captura que “qu” casi siempre va seguido de una vocal); es una entropía de orden 0 (solo frecuencias individuales), no una entropía condicional de orden superior. La extensión opcional de Huffman mencionada en el enunciado no fue implementada.

Punto 10. Simulador cuántico de un qubit (src/cuantica/punto_10.py)

Qué problema resuelve. Simula el estado de un único qubit representado como un vector de dos entradas, permite aplicarle las compuertas X , Z y H , calcula las probabilidades teóricas de medir 0 o 1, y simula 1000 mediciones para comparar la frecuencia observada con la probabilidad teórica.

Idea matemática. Un qubit se representa como una combinación lineal $\alpha|0\rangle + \beta|1\rangle$, codificada como el vector (α, β) . Aplicar una compuerta cuántica es multiplicar ese vector por la matriz correspondiente:

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}, \quad H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}.$$

X es la negación cuántica (intercambia $|0\rangle$ y $|1\rangle$), Z invierte la fase del componente $|1\rangle$, y H (Hadamard) crea una superposición equitativa entre 0 y 1. La probabilidad de medir cada valor es el módulo al cuadrado de la amplitud correspondiente ($|\alpha|^2$ y $|\beta|^2$), que siempre suman 1 porque las matrices usadas son unitarias. La simulación de 1000 mediciones usa esas probabilidades como parámetros de una variable aleatoria discreta (`np.random.choice`), reproduciendo cómo, en la práctica, solo se observan frecuencias relativas y no las probabilidades exactas.

Cómo se ejecuta.

```
python3 src/cuantica/punto_10.py
```

Muestra un menú para escoger la compuerta a aplicar sobre el estado inicial $|0\rangle$.

Qué pruebas se hicieron (los tres casos obligatorios del enunciado):

```

===== X|0> =====
Estado: [0. 1.]
P(0) = 0.0000
P(1) = 1.0000

Después de 1000 mediciones:
0 -> 0
1 -> 1000

===== H|0> =====
Estado: [0.70710678 0.70710678]
P(0) = 0.5000
P(1) = 0.5000

Después de 1000 mediciones:
0 -> 532
1 -> 468

===== HH|0> =====
Estado: [1.00000000e+00 2.23711432e-17]
P(0) = 1.0000
P(1) = 0.0000

Después de 1000 mediciones:
0 -> 1000
1 -> 0

```

Figure 40: prueba 1

```
Compuertas disponibles
1. X
2. Z
3. H
4. H seguida de H

Seleccione una opción: 1

Aplicando X

Estado final:
[0. 1.]

Probabilidad de medir 0: 0.0000
Probabilidad de medir 1: 1.0000

Resultados de 1000 mediciones:
0 -> 0
1 -> 1000
```

Figure 41: prueba 2

```
Compuertas disponibles
1. X
2. Z
3. H
4. H seguida de H

Seleccione una opción: 4

Aplicando H seguida de H

Estado final:
[1.00000000e+00 2.23711432e-17]

Probabilidad de medir 0: 1.0000
Probabilidad de medir 1: 0.0000

Resultados de 1000 mediciones:
0 -> 1000
1 -> 0
```

Figure 42: prueba 3

Limitaciones. Solo se simula un único qubit (no hay entrelazamiento ni sistemas de varios qubits). La simulación asume que las compuertas son exactas y deterministas; el pequeño valor 2.24×10^{-17} en vez de 0 exacto es un error de redondeo de punto flotante, no un fenómeno físico. A diferencia de un computador cuántico real, aquí las probabilidades se conocen de antemano (se calculan directamente del vector de estado) y las “mediciones” son solo un muestreo aleatorio clásico que replica esas probabilidades, no una medición física con ruido de hardware, decoherencia o errores de compuerta.